

Package ‘superdocsr’

August 17, 2026

Title Review-Gated AI Document Editing for Research Papers via SuperDocs

Version 0.1.0

Description A researcher-facing client for the SuperDocs document editing API. Renders an R Markdown paper with 'rmarkdown', uploads it, asks the SuperDocs agent for a targeted edit, and returns the proposed changes for inspection. Nothing is written into the document until the changes are approved explicitly, so an AI edit can never land unreviewed. Long edits are polled with exponential backoff under a wall-clock timeout, and a per-client operation budget stops runaway loops before they spend a quota.

License MIT + file LICENSE

Encoding UTF-8

URL <https://github.com/superdocsapp/superdocs-builds>

BugReports <https://github.com/superdocsapp/superdocs-builds/issues>

Depends R (>= 4.1)

Imports curl, httr2 (>= 1.0.0), jsonlite (>= 1.8.0), tools, utils

Suggests knitr, rmarkdown, testthat (>= 3.0.0), withr

VignetteBuilder knitr

Config/testthat/edition 3

Roxygen list(markdown = TRUE)

Config/roxygen2/version 8.1.0

NeedsCompilation no

Author Aditya Meshram [aut, cre]

Maintainer Aditya Meshram <adime2500@gmail.com>

Contents

superdocsr-package	2
sd_approve	3
sd_awaiting_kind	4
sd_budget	4
sd_cancel	6
sd_changes	6
sd_client	7

sd_continue	8
sd_deny	9
sd_document	10
sd_edit	11
sd_export	12
sd_job	13
sd_knit	14
sd_ops_used	15
sd_review_console	16
sd_upload	17
sd_verify_key	18
sd_wait	18

superdocsr-package	<i>superdocsr: review-gated AI document editing for research papers</i>
--------------------	---

Description

A researcher-facing client for the SuperDocs document editing API. The package covers one workflow end to end: render a paper, upload it, ask for a targeted AI edit, inspect what the AI proposes, approve the parts you agree with, and export the approved document.

Details

The whole point of the package is the gate in the middle. `sd_edit()` defaults to `approval_mode = "ask_every_time"`, so the job pauses and waits; nothing reaches the document until `sd_approve()` names the change ids you accept. There is no function that approves everything for you.

Cost controls

Every client carries an `sd_budget()`. It caps how many billable operations the client may spend and refuses documents above an estimated page count, so a loop that goes wrong stops instead of draining a quota. `small_sample = TRUE` is the cheap preset for a first run.

Testing without a key

The HTTP transport is a plain function stored on the client. Pass your own to `sd_client(transport = )` and the package never touches the network – that is how this package’s own test suite and vignette run.

Author(s)

Maintainer: Aditya Meshram <adime2500@gmail.com>

Authors:

- Aditya Meshram <adime2500@gmail.com>

See Also

Useful links:

- <https://github.com/superdocsr/superdocsr-builds>
- Report bugs at <https://github.com/superdocsr/superdocsr-builds/issues>

sd_approve

*Approve or deny proposed changes***Description**

Sends a decision for the change ids you name to POST /v1/chat/{session_id}/approve. Approved changes are applied atomically; denied ones are discarded, and any feedback goes back to the AI, which may then propose a revision. After the call the job resumes, so poll it again with [sd_wait\(\)](#) – a review can go several rounds.

Usage

```
sd_approve(job, change_ids, approved = TRUE, feedback = NULL)
```

Arguments

job	An sd_job() paused on a change review. Check with sd_awaiting_kind() first; a continue prompt is answered by sd_continue() and rejects an approval with a 409.
change_ids	Character vector of change_id values from sd_changes() .
approved	TRUE to apply them, FALSE to discard them.
feedback	Optional note to the AI, most useful alongside approved = FALSE: say what to do instead and the next proposal reflects it.

Details

change_ids has no default on purpose. There is no "approve everything" switch in this package, because an unreviewed AI edit to a paper you are about to submit is exactly the failure this workflow exists to prevent. To accept a whole batch, say so explicitly: `sd_approve(job, sd_changes(job)$change_id)`.

Value

The refreshed [sd_job\(\)](#), invisibly.

See Also

[sd_deny\(\)](#) for the denying half.

Examples

```
## Not run:
changes <- sd_changes(job)
keep <- changes$change_id[changes$operation == "edit"]
job <- sd_approve(job, keep)
job <- sd_wait(job)

## End(Not run)
```

sd_awaiting_kind

What a paused job is waiting for

Description

awaiting_approval covers two different pauses. A change review wants `sd_approve()`; a large-edit continue prompt wants `sd_continue()`. Sending the wrong one is rejected with a 409, so branch on this first.

Usage

```
sd_awaiting_kind(job)
```

Arguments

job An `sd_job()`.

Value

"continue_prompt", "change_review", or NA when the job is not paused.

Examples

```
client <- sd_client(api_key = "sk_x", transport = function(req) req)
job <- sd_job(client, "j1", "paper-2026", status = "in_progress")
sd_awaiting_kind(job)
```

sd_budget

Cap what a client may spend

Description

A SuperDocs operation is billable; uploads, exports and job polls are not. An edit loop that goes wrong is therefore the only thing that can quietly drain a quota, and this is the object that stops it.

Usage

```
sd_budget(
  max_operations = 25,
  max_pages = NULL,
  chunks_per_page = 12,
  small_sample = FALSE,
  small_sample_pages = 3
)
```

Arguments

max_operations	Maximum billable operations this client may spend. Inf removes the cap.
max_pages	Maximum estimated pages a document may have before an edit is allowed. NULL disables the gate.
chunks_per_page	Chunks-per-page divisor used for the estimate above.
small_sample	If TRUE, apply the cheap preset: at most one operation, at most small_sample_pages pages, and model_tier = "turbo" unless <code>sd_edit()</code> is told otherwise.
small_sample_pages	Page cap used when small_sample = TRUE.

Details

Three controls, all enforced before a request leaves your machine:

- max_operations – the hard stop. Each `sd_edit()` spends one operation from the client's allowance. When the allowance reaches zero the next edit raises an error instead of sending the request.
- max_pages – the size gate. A document above this estimated page count is refused before any edit runs. Uploads are free, so this check costs nothing and happens right after the upload, in `sd_upload()`.
- small_sample – the cheap preset for a first run: one operation, three pages, and the fastest model tier.

Value

An object of class `sd_budget`.

How pages are estimated

The API reports `chunks_count` – the number of addressable blocks (paragraphs, headings, table rows and cells) it parsed out of the file. It does not report a page count, and page count is not knowable from a .docx without rendering it. `max_pages` therefore compares against `ceiling(chunks_count / chunks_per_page)`, an estimate. The default of 12 chunks per page suits a double-spaced manuscript; a dense two-column paper runs higher. Tune `chunks_per_page`, or set `max_pages = NULL` to switch the gate off and rely on `max_operations` alone.

Examples

```
sd_budget()  
sd_budget(small_sample = TRUE)  
sd_budget(max_operations = 3, max_pages = 40)
```

sd_cancel	<i>Cancel a running job</i>
-----------	-----------------------------

Description

Stops the AI mid-edit. Changes already applied stay in the document; pending ones are discarded. Only pending and in_progress jobs can be cancelled.

Usage

sd_cancel(job)

Arguments

job An sd_job().

Value

The refreshed sd_job().

Examples

```
## Not run:
sd_cancel(job)

## End(Not run)
```

sd_changes	<i>Read the changes a job proposes</i>
------------	--

Description

Returns one row per proposed change, so you can look at what the AI wants to do before any of it touches the document.

Usage

sd_changes(job, refresh = TRUE)

Arguments

job An sd_job().
refresh Re-poll the job first. TRUE by default, so the table is never stale.

Value

A data frame of class sd_changes, one row per proposed change, with columns change_id, operation (edit, create or delete), chunk_id, document_id, ai_explanation, old_html, new_html, insert_after_chunk_id, batch_id, batch_total and decided. Zero rows means the AI proposed nothing – an honest answer, not a failure.

The second parse

Proposed changes reach a client by two routes, and they are not shaped the same. `metadata$pending_changes` arrives as ordinary JSON objects. The batched form, in `metadata$intermediate_responses`, arrives with its payload in a content field that is *itself a JSON-encoded string* – the same double encoding the SSE `proposed_change_batch` event uses. Parsing the response once leaves content as a character scalar, and every field read off it comes back NULL. This function calls `jsonlite::fromJSON()` a second time on that string, which is the difference between a populated table and a table of NAs. It is the single most common thing integrators miss, so it is handled here rather than left to the caller.

Both routes are read and merged, deduplicated by `change_id`. Changes already decided before a reconnect (reported in `metadata$pending_batch_decisions`) are marked in the decided column so a review UI can skip them.

See Also

`sd_approve()` to act on the result.

Examples

```
## Not run:
changes <- sd_changes(job)
changes[, c("change_id", "operation", "ai_explanation")]
sd_approve(job, changes$change_id[changes$operation != "delete"])

## End(Not run)
```

sd_client

Create a SuperDocs client

Description

Builds the object every other `sd_*()` function takes. It holds the API key, the base URL, the spending `sd_budget()`, and the HTTP transport.

Usage

```
sd_client(
  api_key = Sys.getenv("SUPERDOCS_API_KEY"),
  base_url = "https://api.superdocs.app",
  budget = sd_budget(),
  transport = NULL,
  timeout = 60,
  max_tries = 3
)
```

Arguments

<code>api_key</code>	SuperDocs API key (the <code>sk_</code> key from Settings > API Keys). Read from the <code>SUPERDOCS_API_KEY</code> environment variable by default. Never type it into a script that goes into version control.
<code>base_url</code>	Base URL of the API. Defaults to <code>https://api.superdocs.app</code> .

budget	An <code>sd_budget()</code> controlling how much this client may spend.
transport	NULL (perform real requests) or a function taking an <code>httr2</code> request and returning an <code>httr2</code> response.
timeout	Per-request timeout in seconds. This is the transport timeout, not the wall-clock budget for an AI edit – that one belongs to <code>sd_wait()</code> .
max_tries	Number of attempts for transient failures (429 and 5xx). Retry-After is honoured when the server sends it.

Details

The transport is a function of one argument – an `httr2` request – that returns an `httr2` response. It defaults to `httr2::req_perform()`. Supplying your own is how you run this package with no network and no API key; the test suite and the package vignette both do exactly that.

Value

An object of class `sd_client`.

Examples

```
# Offline client: a transport that answers every request the same way.
fake <- function(req) {
  httr2::response(
    status_code = 200,
    headers = list(`Content-Type` = "application/json"),
    body = charToRaw("[]")
  )
}
client <- sd_client(api_key = "sk_not_a_real_key", transport = fake)
client

## Not run:
# Real client, key from the environment.
client <- sd_client()
sd_verify_key(client)

## End(Not run)
```

sd_continue

Answer a large-edit continue prompt

Description

A very large edit applies what it can, keeps that work, and pauses to ask whether to carry on. That pause reports status = "awaiting_approval" with `sd_awaiting_kind()` of "continue_prompt", and it is answered here rather than with `sd_approve()` – sending an approval to a continue prompt is rejected with a 409.

Usage

```
sd_continue(job, proceed = TRUE)
```


Arguments

job	An <code>sd_job()</code> paused on a continue prompt.
proceed	TRUE to finish the rest of the edit, FALSE to stop and keep everything applied so far.

Details

Continuing costs more operations, which is exactly why this is a call you make rather than something `sd_wait()` does for you.

Value

The refreshed `sd_job()`.

Examples

```
## Not run:
if (identical(sd_awaiting_kind(job), "continue_prompt")) {
  message(job$metadata$continue_prompt$message)
  job <- sd_continue(job, proceed = TRUE)
}

## End(Not run)
```

sd_deny

*Deny proposed changes***Description**

Shorthand for `sd_approve(approved = FALSE)`. Denying with feedback is the useful shape: the AI reads it and may propose a revision, so keep polling.

Usage

```
sd_deny(job, change_ids, feedback = NULL)
```

Arguments

job	An <code>sd_job()</code> paused on a change review. Check with <code>sd_awaiting_kind()</code> first; a continue prompt is answered by <code>sd_continue()</code> and rejects an approval with a 409.
change_ids	Character vector of change_id values from <code>sd_changes()</code> .
feedback	Optional note to the AI, most useful alongside <code>approved = FALSE</code> : say what to do instead and the next proposal reflects it.

Value

The refreshed `sd_job()`, invisibly.

Examples

```
## Not run:
job <- sd_deny(job, "ch_3", feedback = "Keep the hedge; the effect is not significant.")

## End(Not run)
```

sd_document

A document open in a SuperDocs session

Description

Constructor for the object `sd_upload()` returns. You rarely call this directly; use it when you already hold a session id – to resume work after a crash, say – and want the other verbs to accept it.

Usage

```
sd_document(
  client,
  session_id,
  filename = NA_character_,
  chunks_count = NA_integer_,
  version_id = NA_character_,
  html = NA_character_,
  page_setup = NULL,
  source_path = NA_character_
)
```

Arguments

client	An <code>sd_client()</code> .
session_id	Session holding the document.
filename	Name reported by the API.
chunks_count	Number of addressable blocks the parser found.
version_id	Version identifier reported by the API.
html	Parsed document HTML, when the API returned it.
page_setup	Page geometry list, or NULL for formats that carry none.
source_path	Local file the document came from, if any.

Value

An object of class `sd_document`.

Examples

```
client <- sd_client(api_key = "sk_x", transport = function(req) req)
sd_document(client, session_id = "paper-2026")
```

sd_edit

*Ask SuperDocs to edit a document***Description**

Starts an AI edit against the document's session. By default the job pauses before anything is written: `approval_mode = "ask_every_time"` means the AI proposes changes and waits for `sd_approve()`. Auto-apply exists, but you have to ask for it by name.

Usage

```
sd_edit(
  document,
  instruction,
  async = TRUE,
  approval_mode = c("ask_every_time", "approve_all"),
  model_tier = NULL,
  thinking_depth = NULL,
  response_mode = NULL,
  document_html = NULL,
  client = NULL
)
```

Arguments

document	An <code>sd_document()</code> from <code>sd_upload()</code> , or an <code>sd_job()</code> / session id if you are continuing earlier work.
instruction	What to change, in plain language. Targeted instructions land better than sweeping ones: "tighten the discussion section" beats "improve the paper".
async	TRUE (default) uses POST /v1/chat/async and returns immediately with a job to poll. FALSE uses the synchronous POST /v1/chat, which cannot pause for review and therefore requires <code>approval_mode = "approve_all"</code> .
approval_mode	"ask_every_time" (default) proposes changes and waits. "approve_all" applies them as they are made – appropriate for throwaway drafts, not for a paper you are about to submit.
model_tier	One of "core", "turbo", "pro", "max", or NULL for the server default. <code>small_sample</code> budgets default to "turbo".
thinking_depth	One of "fast", "balanced", "deep", or NULL.
response_mode	"compact" keeps the full document HTML out of every poll and surfaces per-section diffs instead – worth setting for long manuscripts. NULL uses the server default.
document_html	Send document HTML explicitly. Only needed to load or replace the session's document; the server keeps it between turns otherwise. Send it verbatim if you send it at all – stripping <code>data-chunk-id</code> attributes is what breaks targeted editing.
client	An <code>sd_client()</code> . Taken from document when it has one.

Details

This is the one billable call in the package, so it is the one that spends from the client's `sd_budget()`. The spend happens before the request is sent; if the budget refuses, nothing is sent and nothing is charged.

Value

An object of class `sd_job`.

See Also

`sd_wait()` to poll it, `sd_changes()` to read what it proposes.

Examples

```
## Not run:
doc <- sd_upload("manuscript.docx")
job <- sd_edit(doc, "Tighten the abstract to 150 words without losing the result.")
job <- sd_wait(job)
sd_changes(job)

## End(Not run)
```

sd_export

Export a document to a file

Description

Renders the session's current document through POST `/v1/documents/export` and writes the bytes to disk. Exports are not billable, so exporting often is free – do it after every approved round rather than at the end.

Usage

```
sd_export(
  document,
  format = "docx",
  path = NULL,
  options = list(),
  client = NULL
)
```

Arguments

document	An <code>sd_document()</code> , an <code>sd_job()</code> , or a session id.
format	One of "docx" (default), "pdf", "html", "markdown", "txt". RTF is an upload format only; it is not an export target.
path	Where to write the file. Defaults to the document's name with the format's extension, in the working directory.
options	Named list of export options passed through to the API: <code>paper_size</code> , <code>orientation</code> , <code>margins</code> , <code>custom_margins_inches</code> , <code>filename</code> , <code>embed_images</code> , <code>watermark_text</code> , <code>watermark_opacity</code> .
client	An <code>sd_client()</code> . Taken from document when it has one.

Details

What gets exported is what is *in* the document. Changes still awaiting approval are not in it, and this function says so rather than letting a successful-looking export imply otherwise.

Value

The path written, invisibly. Raises an error if the response was empty, so a returned path always means a real file.

Examples

```
## Not run:
sd_export(doc, "docx", "manuscript-revised.docx")
sd_export(doc, "pdf", "manuscript-revised.pdf",
  options = list(paper_size = "A4", margins = "narrow")
)

## End(Not run)
```

sd_job

A SuperDocs async job

Description

Constructor for the object `sd_edit()` returns. Build one yourself to resume polling a `job_id` you persisted – after an R session restart, for instance. Jobs are deleted one hour after they are created, so a job id older than that is gone rather than merely slow.

Usage

```
sd_job(
  client,
  job_id,
  session_id,
  status = "pending",
  progress = NA_real_,
  result = NULL,
  error = NULL,
  metadata = NULL,
  job_type = NA_character_
)
```

Arguments

<code>client</code>	An <code>sd_client()</code> .
<code>job_id</code>	Job identifier. Opaque; treat it as a string.
<code>session_id</code>	Session the job belongs to.
<code>status</code>	One of pending, in_progress, awaiting_approval, completed, failed, cancelled.
<code>progress</code>	Progress percentage reported by the API.
<code>result</code>	Result list, present once the job completes.

error	Error string, present when the job failed.
metadata	Metadata list, carrying pending_changes, awaiting_kind, continue_prompt and intermediate_responses.
job_type	Job type reported by the API.

Value

An object of class `sd_job`.

Examples

```
client <- sd_client(api_key = "sk_x", transport = function(req) req)
sd_job(client, job_id = "550e8400", session_id = "paper-2026")
```

sd_knit

Render an R Markdown paper and edit it through SuperDocs

Description

Calls `rmarkdown::render()` on `input_rmd`, uploads the rendered file, asks for instruction, waits for the proposal, hands it to review, applies exactly the decisions review returns, and optionally exports the result.

Usage

```
sd_knit(
  input_rmd,
  instruction,
  output_format = "word_document",
  client = sd_client(),
  review = sd_review_console,
  export_path = NULL,
  export_format = NULL,
  timeout = 300,
  max_rounds = 3L,
  continue = FALSE,
  render_args = list(),
  ...
)
```

Arguments

input_rmd	Path to the .Rmd file.
instruction	What to change, in plain language.
output_format	Passed to <code>rmarkdown::render()</code> . "word_document" by default, which gives SuperDocs the .docx it round-trips with the highest fidelity.
client	An <code>sd_client()</code> .
review	Function taking the <code>sd_changes()</code> data frame and returning the <code>change_id</code> values to approve. Return <code>character(0)</code> to deny everything, or <code>NULL</code> to stop and leave the review open for a human.

export_path	Where to write the approved document. NULL skips the export; nothing is exported when nothing was approved.
export_format	Format for the export. Inferred from export_path's extension when possible, otherwise "docx".
timeout	Seconds to wait on each poll, passed to <code>sd_wait()</code> .
max_rounds	Maximum review rounds before giving up and leaving the job for a human.
continue	Answer given to a large-edit continue prompt: FALSE (default) stops and keeps what is applied, TRUE finishes the edit.
render_args	Extra arguments for <code>rmarkdown::render()</code> .
...	Passed to <code>sd_edit()</code> , e.g. <code>model_tier = "max"</code> .

Details

The review step is a function you supply, and there is no default that approves anything. `sd_review_console()`, the default, asks a human in an interactive session and refuses to guess in a script. Every change gets an explicit decision – ids you name are approved, the rest are denied – so the job never sits blocking the session waiting on a review nobody is going to give.

Value

An object of class `sd_knit_result`: a list with `rendered`, `document`, `job`, `changes`, `approved`, `denied` and `export_path`.

Stopping rules

Two loops live here, and both are bounded. `max_rounds` caps how many times a denied-with-feedback round trip may repeat, because the AI is entitled to keep proposing revisions and each one costs. `continue` decides what happens when a large edit pauses to ask whether to keep going; it defaults to FALSE, which keeps the work already applied and stops, rather than spending more without being asked. On top of both, the client's `sd_budget()` is the hard ceiling.

Examples

```
## Not run:
result <- sd_knit(
  "paper.Rmd",
  "Tighten the discussion section; keep every citation exactly as written.",
  export_path = "paper-revised.docx"
)
result$changes

## End(Not run)
```

sd_ops_used

Operations spent and left on a client

Description

Operations spent and left on a client

Usage

```
sd_ops_used(client)
```

```
sd_ops_remaining(client)
```

Arguments

client An `sd_client()`.

Value

A single number: operations spent so far, or operations still available (possibly Inf).

Examples

```
client <- sd_client(api_key = "sk_x", transport = function(req) req)
sd_ops_used(client)
sd_ops_remaining(client)
```

sd_review_console	<i>Review helpers for sd_knit()</i>
-------------------	-------------------------------------

Description

`sd_review_console()` prints each proposed change and asks a human which to approve. In a non-interactive session it refuses to decide and raises an error, because the alternative – guessing – is the behaviour this package exists to avoid.

Usage

```
sd_review_console(changes)
```

```
sd_review_none(changes)
```

Arguments

changes The `sd_changes()` data frame.

Details

`sd_review_none()` returns NULL, which runs the workflow up to the review and stops there, leaving the job open for a person. It is the right choice for a scheduled script that should prepare work rather than commit it.

Value

A character vector of `change_id` values to approve, `character(0)` to deny everything, or NULL to leave the review open.

Examples

```
client <- sd_client(api_key = "sk_x", transport = function(req) req)
job <- sd_job(client, "j1", "paper-2026")
sd_review_none(sd_changes(job, refresh = FALSE))
```


sd_upload

*Upload a document into a SuperDocs session***Description**

Sends a file to POST /v1/documents/upload and loads it as the session's active editable document. The file is streamed from disk by curl, not read into R first, so a large manuscript does not have to fit in memory twice.

Usage

```
sd_upload(
  path,
  client = sd_client(),
  session_id = NULL,
  open_mode = c("replace", "new_focused", "background")
)
```

Arguments

path	Path to the file. One of .pdf, .docx, .txt, .rtf, .md, .html, .htm. Legacy binary .doc is not accepted – convert it to .docx first.
client	An sd_client() .
session_id	Session to load the document into. Generated if omitted. May contain only letters, digits, _, - and ., up to 256 characters.
open_mode	"replace" (the default) swaps the session's focused document. "new_focused" opens the file as an extra document and focuses it; "background" opens it without stealing focus.

Details

Uploading is not a billable operation. The page gate from [sd_budget\(\)](#) is applied here, right after the parse, because that is the first moment the real chunk count is known and it is still before anything costs money.

Value

An object of class sd_document.

Examples

```
## Not run:
client <- sd_client()
doc <- sd_upload("manuscript.docx", client)
doc

## End(Not run)
```

sd_verify_key	<i>Confirm an API key works</i>
---------------	---------------------------------

Description

Calls GET /v1/sessions, which the SuperDocs documentation names as the cheapest way to check an sk_key. It is not billable and it does not create anything. GET /v1/users/me is deliberately not used here: it is web-app-only and returns 401 even for a valid API key.

Usage

```
sd_verify_key(client)
```

Arguments

client	An <code>sd_client()</code> .
--------	-------------------------------

Value

TRUE, invisibly, if the key is accepted. Errors otherwise.

Examples

```
## Not run:  
sd_verify_key(sd_client())  
  
## End(Not run)
```

sd_wait	<i>Wait for a job to reach a stopping point</i>
---------	---

Description

Polls GET /v1/jobs/{job_id} until the job stops needing to be polled, and returns it. Four things count as a stopping point: completed, failed, cancelled, and awaiting_approval – the last one being the review gate, which is a result, not a delay.

Usage

```
sd_wait(  
  job,  
  timeout = 300,  
  backoff = TRUE,  
  interval = 2,  
  max_interval = 30,  
  verbose = interactive()  
)
```

Arguments

job	An <code>sd_job()</code> .
timeout	Wall-clock seconds to wait before giving up.
backoff	If TRUE (default), the gap between polls grows by half each time, up to <code>max_interval</code> . If FALSE, poll every <code>interval</code> seconds.
interval	First gap between polls, in seconds.
max_interval	Longest gap between polls, in seconds.
verbose	Report progress while waiting. Defaults to on in an interactive session, off in a script.

Details

An edit on a real paper takes anywhere from ten seconds to several minutes with nothing visible happening in between. That is the API working, not a crash. The default timeout of 300 seconds suits a multi-section edit; a full-document restructure of a long manuscript wants more. The server caps any single turn at 30 minutes, and jobs are deleted an hour after creation, so there is no point waiting longer than that.

Value

The updated `sd_job()`. Raises `sd_job_failed` if the job failed or was cancelled, and `sd_timeout_error` if timeout runs out first – the job id stays valid in that case, so you can call `sd_wait()` again.

Examples

```
## Not run:
job <- sd_wait(sd_edit(doc, "Rewrite the conclusion."), timeout = 600)
sd_awaiting_kind(job)

## End(Not run)
```